

To flash MicroPython firmware binary on ESP8266 board, a handy command line tool known as `esptool` can be used. It can be downloaded directly from Python package manager using the following commands:

```
sudo pip install esptool
Check for the serial port after
connecting the board it is /dev/tty.
SLAB_USBtoUART in our case
ls /dev/tty* can help you identify it.
esptool.py --port "serial_port" erase_
flash
esptool.py --port "serial_port" --baud
460800 write_flash --flash_size=detect 0
"firmware_file"
```

Here, .bin firmware file should be kept in the current working directory:

```
iAyan:~ iAyan$ sudo esptool.py --port /
dev/tty.SLAB_USBtoUART erase_flash
esptool.py v1.2.1
Connecting...
Running Cessanta flasher stub...
Erasing flash (this may take a while)...
Erase took 10.8 seconds
iAyan:~ iAyan$ esptool.py --port /
dev/tty.SLAB_USBtoUART --baud 460800
write_flash --flash_size=detect 0 esp8266-
20161110-v1.8.6.bin
esptool.py v1.2.1
Connecting...
Auto-detected Flash size: 32m
Running Cessanta flasher stub...
Flash params set to 0x0040
Writing 569344 @ 0x0... 569344 (100 %)
Wrote 569344 bytes at 0x0 in 14.1
seconds (324.1 kbit/s)...
Leaving...
iAyan:~ iAyan$
```

Using the serial REPL

Just like flashing MicroPython to the board, its REPL can be accessed over the serial port. Simply connect your MicroPython compatible board, ESP8266 board in this case, and it will mount as a serial device (COM-Mxx in Windows and `/dev/ttySLAB` in OSX and Linux). Use any serial emulator like Putty on Windows, or minicom or screen on Linux/OSX to access the serial REPL, using the



Fig. 5: NodeMCU

baud rate of 115,200, as follows:

```
sudo screen /dev/tty.SLAB_USBtoUART
115200
OR
sudo minicom -D /dev/ttySLAB_USBtoUART
```

For Windows, use Putty, for which the serial to be used is COM-Mxx (xx = COMM port number under Device Manager) and the baud rate is 115,200.

If everything goes well, you will see the familiar Python prompt, at which point you can write Python commands and view the immediate outputs. Type `'print ("Hello World!")'`, and yes, it is the same old Python that you love.

```
sudo screen /dev/tty.SLAB_USBtoUART
115200
"press enter"
>>> print("Hello, MicroPython!")
Hello, MicroPython!
>>> 2+5
7
>>>
```

Using WebREPL

One unique feature of MicroPython on the boards that support networking (like ESP8266) is a WebREPL (REPL like a Python command line) that is accessible through a Web page. Instead of using a serial connection to the board, you can run Python code directly from your browser in a simple terminal. You do not even need to connect the board to a Wi-Fi network; it can create its own network, which you can use to access WebREPL. The new releases of MicroPython come with WebREPL disabled by default, so use the following commands to

enable it over serialREPL:

```
>>> import webrepl_setup
WebREPL daemon auto-start status:
disabled
Would you like to (E)nable or (D)isable
it running on boot?
(Empty line to quit)
> E
To enable WebREPL, you must set
password for it
New password: python
Confirm password: python
Changes will be activated after reboot
Would you like to reboot now? (y/n) y
```

Once enabled, ESP8266 will start in AP mode and create a hotspot. You can connect to it with the password `micropython` and access WebREPL using the IP `ws://192.168.4.1:8266/`. Enter the password that you set earlier. WebREPL client can be accessed at <http://micropython.org/webrepl/> or downloaded from GitHub (<https://codecademy.com/micropython/webrepl/zip/master>).

Tools to make life easy

Adafruit MicroPython Tool (Ampy). This utility is used to interact with MicroPython board over a serial connection. Ampy is a simple command line tool to manipulate files and run code on MicroPython board over its serial connection. With it, you can send files from your computer to a MicroPython board's file system, download files from a board to your computer and send a Python script to a board to be executed. Download Ampy using Python Pip.

```
sudo pip3 install adafruit-ampy
Transfer files from a PC to the
board using Ampy and vice versa,
as follows:
ampy --port "serial_port" put "file_name.
py"
ampy --port "serial_port" get "file_name.
py"
iAyan:~ iAyan$ ampy --port /dev/tty.
SLAB_USBtoUART get boot.py
# This file is executed on every boot
```